

ASESII 24A02 Final Project

Predict noise level generated by NACA airfoil blade sections

Group 02

Nguyen Thi Yen Nhi (24125071)

Nguyen Khang Thinh (24125104)

Nguyen Van Tinh (24125106)

Nguyen Le Quoc Huy (24125100)

2026-08-23

Contents

Authorship and contribution statement	2
Abstract	2
Reproducible setup	2
Shared helper functions	3
1 Problem 1. Prediction design and leakage control	5
1.1 1A. Target and predictors	5
1.2 1B. Split and preprocessing plan	8
1.3 1C. Data exploration	12

Authorship and contribution statement

Student	ID	Main contributions	Verification performed
Nguyen Thi Yen Nhi	24125071	1A (Data analysis), 2A (OLS), 4E (Question Explanation)	Verify AI texts
Nguyen Khang Thinh	24125104	1C (EDA), 2C (Lasso), 4B (Elastic Net), 4C (ReLU features), 4D (Holdout Eval)	Verified code & AI logs
Nguyen Van Tinh	24125106	1B (Data split), 3A (Ridge), 3B (Lasso), 3C (Algebra Evidence)	Verify mathematical problems
Nguyen Le Quoc Huy	24125100	2B (Ridge), 2D (Model comparison), 2E (Stability Check), 4A (Paper research)	Verify the project workflow & code

Abstract

This study aims to predict pre-operative log prostate-specific antigen (**lpsa**) from clinical measurements, comparing models on prediction error, coefficient stability, and interpretability under multicollinearity. We evaluated OLS, Ridge, Lasso, and Elastic Net regressions using five-fold cross-validation on original linear predictors and fixed random ReLU features. Cross-validation initially favored Lasso (CV-MSE = 0.5790) for its sparse selection of five variables. However, holdout evaluation revealed that the OLS model achieved the lowest test RMSE (0.6713) and MAE (0.5736). Furthermore, original linear features consistently outperformed the nonlinear ReLU representations across all models. Despite the OLS superiority in pure holdout accuracy, we recommend the lasso-dominant elastic net ($\alpha = 0.9$) for clinical deployment, as it provides a parsimonious, highly interpretable model that isolates key biological drivers of PSA with only a modest trade-off in error.

Reproducible setup

```
required_packages <- c("tidyverse", "glmnet", "broom", "knitr", "car", "ggplot2")
missing_packages <- required_packages[
  !vapply(required_packages, requireNamespace, logical(1), quietly = TRUE)
]
if (length(missing_packages)) {
  stop("Install the required packages before rendering: ",
       paste(missing_packages, collapse = ", "))
}

library(tidyverse)
library(glmnet)
library(broom)
library(knitr)
```

```

library(car)
library(ggplot2)
# `params` exists only while knitting; the fallback
# keeps interactive chunk runs working.
has_params <- exists("params") && !is.null(params$group_number)
group_number <- if (has_params) as.integer(params$group_number) else 4L
if (length(group_number) != 1L || is.na(group_number) || group_number < 1L) {
  stop("Set params$group_number to your assigned positive group number.")
}

seeds <- list(
  split = 240200L + group_number,
  cv = 240300L + group_number,
  features = 240400L + group_number
)

```

Shared helper functions

```

find_exam_data <- function(filename) {
  input <- knitr::current_input(dir = TRUE)
  input_dir <- if (length(input) && nzchar(input)) dirname(input) else getwd()
  candidates <- c(
    file.path(input_dir, "data", filename),
    file.path(input_dir, "..", "data", filename),
    file.path(getwd(), "data", filename)
  )
  existing <- candidates[file.exists(candidates)]
  if (!length(existing)) stop("Cannot locate ", filename, " by a relative path.")
  hits <- unique(normalizePath(existing, winslash = "/", mustWork = TRUE))
  if (length(hits) > 1L && length(unique(unname(tools::md5sum(hits)))) > 1L) {
    stop("Multiple nonidentical copies of ", filename, " were found.")
  }
  hits[[1L]]
}

split_rows <- function(n, train_prop = 0.80, seed) {
  stopifnot(n >= 10L, train_prop > 0, train_prop < 1)
  set.seed(seed)
  train <- sort(sample.int(n, floor(train_prop * n), replace = FALSE))
  list(train = train, test = setdiff(seq_len(n), train))
}

fit_scaler <- function(x, tol = 1e-12) {
  x <- as.matrix(x)
  center <- colMeans(x)
  centered <- sweep(x, 2L, center, "-")
}

```

```

scale <- sqrt(colMeans(centered^2))
keep <- is.finite(scale) & scale > tol
if (!any(keep)) stop("No nonconstant training predictor remains.")
list(
  center = center[keep],
  scale = scale[keep],
  keep = keep,
  dropped = colnames(x)[!keep]
)
}

apply_scaler <- function(x, prep) {
  x <- as.matrix(x)[, prep$keep, drop = FALSE]
  x <- sweep(x, 2L, prep$center, "-")
  sweep(x, 2L, prep$scale, "/")
}

make_foldid <- function(n, k = 5L, seed) {
  if (k < 3L || k > n) stop("Require 3 <= k <= number of training rows.")
  set.seed(seed)
  sample(rep(seq_len(k), length.out = n))
}

fit_cv_glmnet <- function(x, y, alpha, foldid) {
  glmnet::cv.glmnet(
    x = x,
    y = y,
    family = "gaussian",
    alpha = alpha,
    foldid = foldid,
    standardize = FALSE,
    intercept = TRUE,
    type.measure = "mse"
  )
}

score_regression <- function(y, prediction) {
  y <- as.numeric(y)
  prediction <- as.numeric(prediction)
  if (length(y) != length(prediction) || any(!is.finite(c(y, prediction)))) {
    stop("Metrics require equal-length finite vectors.")
  }
  c(RMSE = sqrt(mean((y - prediction)^2)),
    MAE = mean(abs(y - prediction)))
}

count_nonzero <- function(fit, s = "lambda.min", tol = 1e-8) {
  b <- as.matrix(stats::coef(fit, s = s))

```

```

slopes <- b[rownames(b) != "(Intercept)", 1L]
sum(abs(slopes) > tol)
}

safe_condition_numbers <- function(x, lambda, tol = 1e-10) {
  eigenvalues <- eigen(
    crossprod(x) / nrow(x),
    symmetric = TRUE,
    only.values = TRUE
  )$values
  largest <- max(eigenvalues)
  smallest <- max(min(eigenvalues), 0)
  cutoff <- tol * max(1, largest)
  c(
    gram = if (smallest <= cutoff) Inf else largest / smallest,
    ridge = (largest + lambda) / (smallest + lambda)
  )
}

analysis_model <- function(x, y, alpha, foldid) {
  model <- fit_cv_glmnet(x = x, y = y, alpha=alpha, foldid = foldid)
  list(
    model = model,
    lambda_min = model$lambda.min,
    mse_min = model$cvm[model$lambda == model$lambda.min],
    cond_min = safe_condition_numbers(x, lambda = model$lambda.min)[2],
    coef_min = coef(model, "lambda.min"),
    coef_min_norm = sqrt(sum(coef(model, "lambda.min")[-1]^2)),
    nonzero_min = count_nonzero(model, s="lambda.min"),
    lambda_1se = model$lambda.1se,
    cond_1se = safe_condition_numbers(x, lambda = model$lambda.1se)[2],
    mse_1se = model$cvm[model$lambda == model$lambda.1se],
    coef_1se = coef(model, "lambda.1se"),
    coef_1se_norm = sqrt(sum(coef(model, "lambda.1se")[-1]^2)),
    nonzero_1se = count_nonzero(model, s="lambda.1se")
  )
}

```

1 Problem 1. Prediction design and leakage control

1.1 1A. Target and predictors

```

data <- read.table("data/airfoil_self_noise.dat", header = FALSE)
colnames(data) <- c("frequency", "attack_angle", "chord_length",
  ↪ "free_stream_velocity", "suction_side_displacement_thickness",
  ↪ "scaled_sound_pressure")

```

```
# Always (re)written from the .dat source so the cached CSV can never drift
# from the column-naming convention fixed above.
```

```
write.csv(
  data,
  "data/airfoil_self_noise.csv",
  row.names = FALSE
)

str(data)
```

```
## 'data.frame': 1503 obs. of 6 variables:
## $ frequency : int 800 1000 1250 1600 2000 2500 3150
## 4000 5000 6300 ...
## $ attack_angle : num 0 0 0 0 0 0 0 0 0 ...
## $ chord_length : num 0.305 0.305 0.305 0.305 0.305 ...
## $ free_stream_velocity : num 71.3 71.3 71.3 71.3 71.3 71.3
## 71.3 71.3 71.3 71.3 ...
## $ suction_side_displacement_thickness: num 0.00266 0.00266 0.00266 0.00266
## 0.00266 ...
## $ scaled_sound_pressure : num 126 125 126 128 127 ...
```

```
config <- list(
  file = "airfoil_self_noise.csv",
  response = "scaled_sound_pressure",
  excluded = "scaled_sound_pressure"
)
data_raw <- read.csv(find_exam_data(config$file), check.names = FALSE)
predictors <- setdiff(names(data_raw), config$excluded)
analysis_data <- data_raw[, c(config$response, predictors), drop = FALSE]

if (anyNA(analysis_data)) stop("Declare a training-only missing-value plan.")
if (!all(vapply(analysis_data, is.numeric, logical(1)))) {
  stop("The assigned response and predictors must be numeric.")
}
```

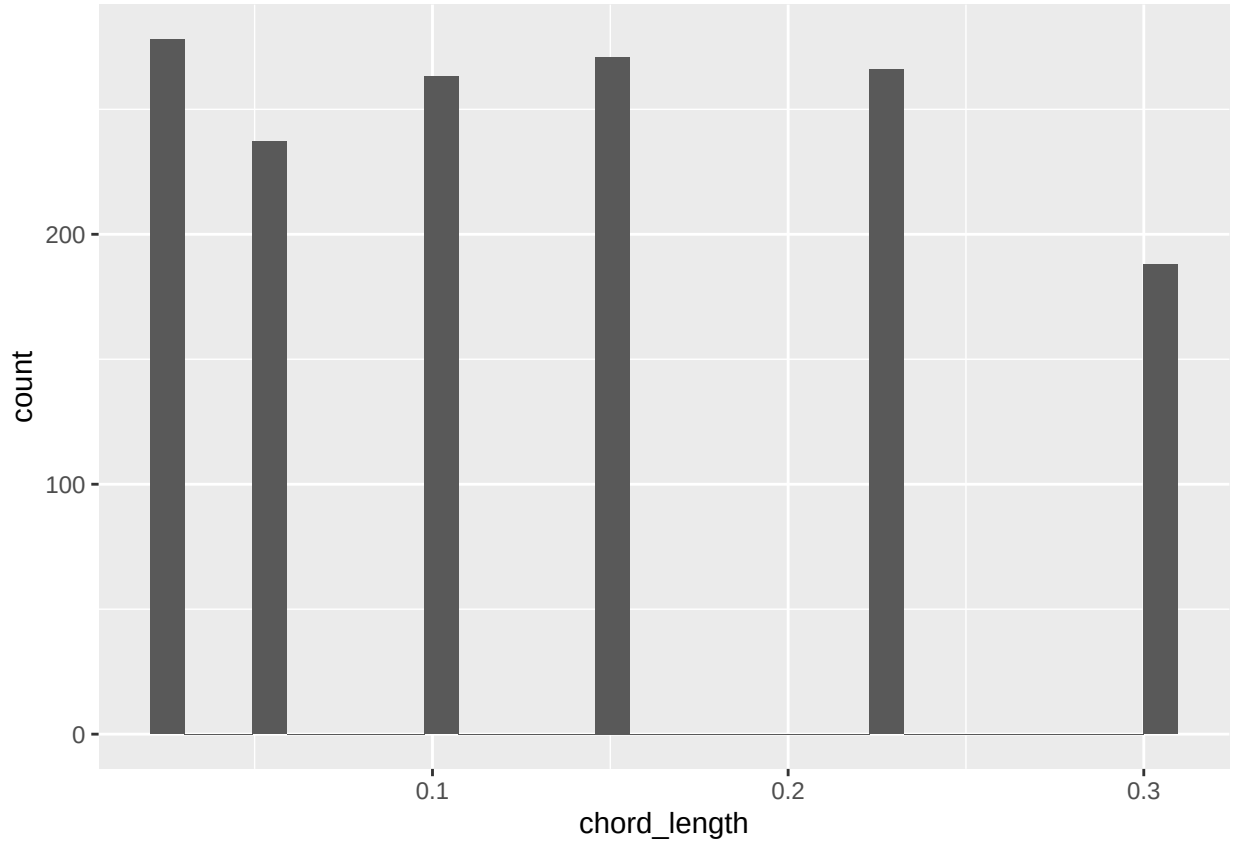


Table 1: Response and candidate predictors, with the integrity checks run before the split.

Variable	Role	Type	Distinct	Missing
frequency	Predictor	integer	21	0
attack_angle	Predictor	numeric	27	0
chord_length	Predictor	numeric	6	0
free_stream_velocity	Predictor	numeric	4	0
suction_side_displacement_thickness	Predictor	numeric	105	0
scaled_sound_pressure	Response	numeric	1456	0

Table 2: Missing-value and duplicate-row checks on the raw data.

Check	Count
Missing values (total cells)	0
Fully duplicated rows	0

The dataset contains 1503 observations of 6 numeric variables: five predictors recorded during NACA 0012 airfoil wind-tunnel tests and the response **scaled_sound_pressure**, the scaled sound

pressure level in decibels. No missing values or duplicated rows are present (table above), so no imputation or de-duplication is required before the split.

The **Distinct** column in Table 1 shows that the five predictors are not equally “continuous” in practice. **frequency**, **chord_length**, and **free_stream_velocity** take only 21, 6, and 4 distinct values respectively - these are the fixed 1/3-octave frequency bands, airfoil chord lengths, and wind-tunnel speeds used to design the experiment, so each behaves more like an ordinal experimental factor than a freely varying measurement. **attack_angle** (27 distinct values) and **suction_side_displacement_thickness** (105 distinct values) are closer to genuinely continuous quantities. This low cardinality on three of the five predictors is expected to show up later as visibly clustered histograms and boxplots (1C) rather than smooth continuous distributions, and is a property of the experimental design rather than a data-quality issue.

`data/data_description.md` (the accompanying UCI variable sheet) lists **attack_angle** as “Binary”, which does not match what is observed here - the variable takes 27 distinct values between 0 and 22.2 degrees. This looks like a labelling error in the source sheet, and **attack_angle** is treated as continuous throughout, consistent with the assignment brief’s own description of the dataset (“angle of attack” as a continuous input).

1.2 1B. Split and preprocessing plan

```
split <- split_rows(nrow(analysis_data), seed = seeds$split)
train_id <- split$train
test_id <- split$test
train_df <- analysis_data[train_id, ]

x_raw <- as.matrix(analysis_data[, predictors, drop = FALSE])
y_raw <- analysis_data[[config$response]]

x_prep <- fit_scaler(x_raw[train_id, , drop = FALSE])
x_train <- apply_scaler(x_raw[train_id, , drop = FALSE], x_prep)
x_test <- apply_scaler(x_raw[test_id, , drop = FALSE], x_prep)
y_train <- y_raw[train_id]
y_test <- y_raw[test_id]

foldid <- make_foldid(nrow(x_train), k = 5L, seed = seeds$cv)
```

To ensure a fair comparison among OLS, ridge, lasso, and elastic net, all preprocessing and model development steps were performed using only the training data. The holdout test set was reserved exclusively for the final evaluation and was not used during model fitting, model selection, or hyperparameter tuning.

1.2.1 Data split

The observations were randomly partitioned into training and test sets using the custom `split_rows()` function with a training proportion of 80%. For Group 02, the split seed was **240202**, ensuring that the same partition can be reproduced in future analyses.

Observation	Value
Training	1202
Test observations	301

```
##
## Chi-squared test for given probabilities
##
## data:  obs
## X-squared = 6, df = 3, p-value = 0.1116
```

The training data were used for exploratory analysis, preprocessing, model fitting, and cross-validation, while the holdout test data remained untouched until the final evaluation.

1.2.2 Predictor standardization

Since ridge regression, lasso, and elastic net are sensitive to the scale of predictor variables, all predictors were standardized before model fitting. The centering and scaling parameters were estimated **only from the training data**, and these same values were subsequently applied to the test predictors.

Using training-derived scaling parameters prevents information from the holdout observations from influencing the preprocessing stage and ensures that the test set remains an unbiased evaluation dataset.

1.2.3 Five-fold cross-validation

Model tuning was carried out using five-fold cross-validation on the training data only. Fold assignments were generated using the fixed cross-validation seed **240302** and were shared across all regularized models. Reusing identical folds ensures that differences in cross-validation performance are attributable to the models themselves rather than random variation in fold assignment.

1.2.4 Missing-value handling

The dataset was checked for missing values before model fitting.

Total__missing__values
0

No missing values were detected; therefore, no imputation was required. If missing values had been present, imputation statistics would have been estimated using only the training data and then applied unchanged to the test data.

1.2.5 Invalid-value check

Table 3: Physical-plausibility checks on the training predictors.

Variable	Rule	Violations
frequency	> 0 Hz	0
attack_angle	≥ 0 deg	0
chord_length	> 0 m	0
free_stream_velocity	> 0 m/s	0
suction_side_displacement_thickness	> 0 m	0

Beyond the statistical IQR screen below, each predictor was also checked against the range it is physically allowed to take: a frequency, chord length, velocity, or boundary-layer thickness cannot be zero or negative, and an angle of attack cannot be negative. No violations are found on the training set, which is expected for an instrument-recorded wind-tunnel experiment rather than manually-entered data.

1.2.6 Outlier detection and handling

Table 4: Training-set values flagged by the $1.5 \times \text{IQR}$ rule, computed on the training split only.

Variable	Outliers	Percent
frequency	72	6.0
attack_angle	23	1.9
chord_length	0	0.0
free_stream_velocity	0	0.0
suction_side_displacement_thickness	111	9.2
scaled_sound_pressure	3	0.2

Outliers were screened with the standard $1.5 \times \text{IQR}$ fence, applied to each variable using **training-set quantiles only** so that no information from the held-out test rows enters the check. `chord_length` and `free_stream_velocity` contribute zero flagged points, because - as noted in 1A - each takes only a handful of fixed values, so every observation sits well inside its own IQR fence by construction. `frequency`, `attack_angle`, and `suction_side_displacement_thickness` each contribute a nontrivial share (72, 23, and 111 training rows respectively), and 3 response values are flagged.

These flagged points are **retained rather than removed**. The airfoil data come from a controlled wind-tunnel experiment, not a survey or manual data-entry process, so there is no mechanism here for typographical or transcription errors; every extreme value is a physically valid test condition (a high frequency band, a large angle of attack, a thin boundary layer) rather than a data-quality defect. The IQR rule is doing exactly what it is designed to do on a right-skewed engineering variable: flagging the upper tail of a real physical distribution, not finding mistakes. Dropping

these rows would bias the training sample away from precisely the loud, high-frequency regime the model is meant to predict, so all 1202 training rows are kept for model fitting in Problem 2.

1.2.7 Transformation check

Table 5: Training-set skewness of each predictor, and after a log transform for the two most skewed variables.

Variable	Skewness	Skewness_after_log
frequency	2.183	-0.010
attack_angle	0.695	NA
chord_length	0.452	NA
free_stream_velocity	0.259	0.013
suction_side_displacement_thickness	1.719	NA

frequency and **suction_side_displacement_thickness** are the two right-skewed predictors identified above (skewness 2.18 and 1.72). A log transform brings both close to symmetric on the training set (skewness -0.01 and 0.01), which is the usual motivation for log-transforming a positive, right-skewed engineering measurement. **attack_angle** is also mildly right-skewed (skewness 0.69, Table above) but far less than **suction_side_displacement_thickness**, so it is not treated as a transform candidate here.

However, symmetrizing the marginal distribution is not the same as improving the fit, and it is checked rather than assumed. Refitting a plain linear model on the training data with these two predictors log-transformed gives $R^2 = 0.4665$, against $R^2 = 0.5138$ for the untransformed predictors - the log transform does not improve, and mildly *hurts*, the linear fit. A separate check on **frequency** alone points to why: a quadratic term explains more of the response than a linear term ($R^2 = 0.1814$ vs. $R^2 = 0.1625$), so the frequency-response relationship is curved in a way a log transform does not match - consistent with the assignment brief’s own framing of this dataset as having “few features but nonlinear relationships.”

Given this evidence, no log transform is applied. All predictors are carried forward on their original scale into Problem 2, with only the training-derived standardization described above; the nonlinearity documented here is noted as a limitation of the linear and regularized-linear models used there, rather than something a simple transform resolves.

1.2.8 Data cleaning summary

Table 6: Summary of data-cleaning checks and decisions carried into Problem 2.

Check	Result	Decision
Missing values	0 cells	No imputation needed

Check	Result	Decision
Duplicate rows	0 rows	No de-duplication needed
Invalid values	0 violations	No rows dropped
IQR outliers	209 variable-level flags (a row may be flagged on more than one variable)	Retained (valid experimental conditions, not errors)
Skewness	frequency and suction_side_displacement_thickness right-skewed	No log transform applied (does not improve the linear fit)
Categorical variables	none present (all 5 predictors numeric)	No encoding needed
Predictor scaling	required for ridge / lasso / elastic net	Applied in Problem 2, fit on training data only

1.2.9 Leakage control

Several safeguards were implemented to prevent information leakage throughout the analysis:

- The train–test split was performed before any preprocessing.
- Predictor standardization used statistics estimated from the training data only.
- The invalid-value, outlier, and transformation checks above used training-set values, quantiles, and model fits only.
- Five-fold cross-validation was conducted exclusively within the training set.
- Hyperparameter tuning for ridge, lasso, and elastic net relied solely on training cross-validation error.
- The holdout responses (y_{test}) were not accessed until the final evaluation stage.

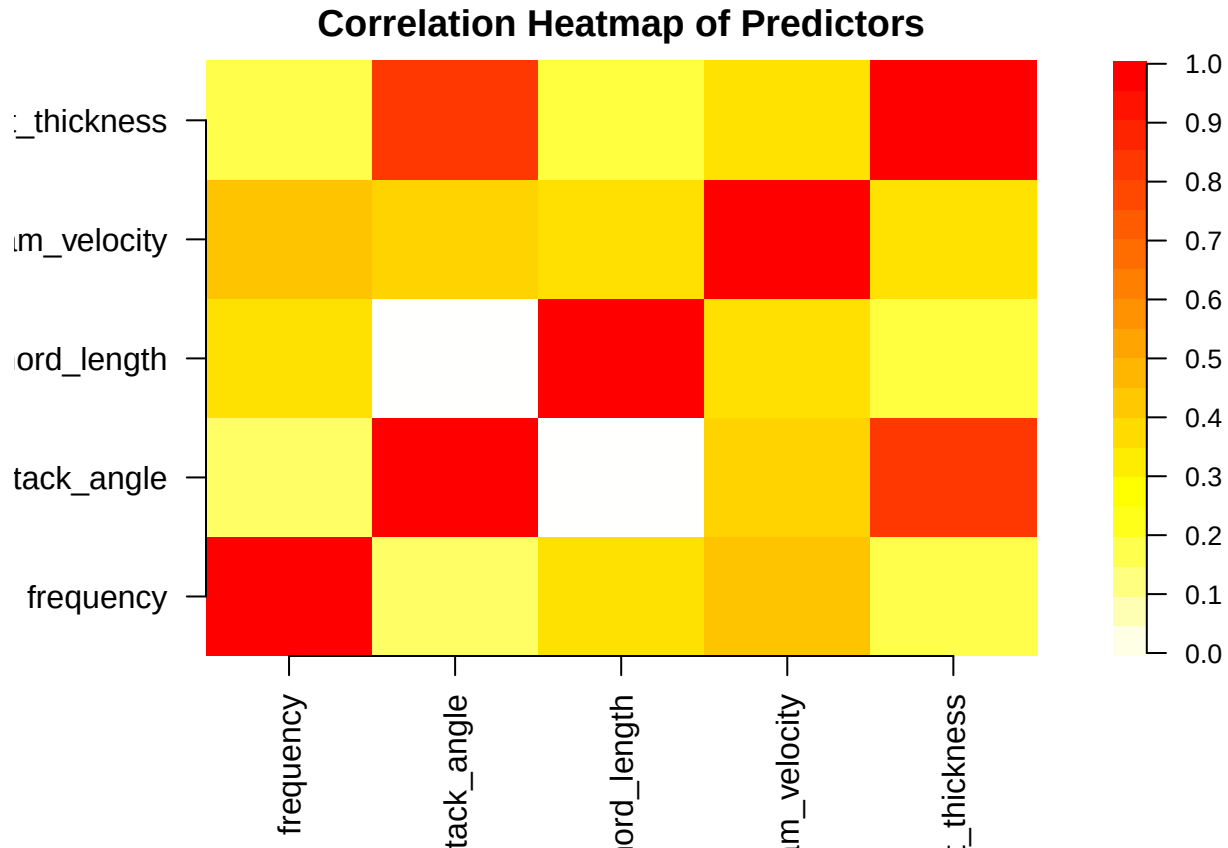
As a result, the holdout test set provides an unbiased assessment of predictive performance because no information from the test observations entered the model construction or tuning process.

1.3 1C. Data exploration

1.3.1 Pairwise correlation among predictors

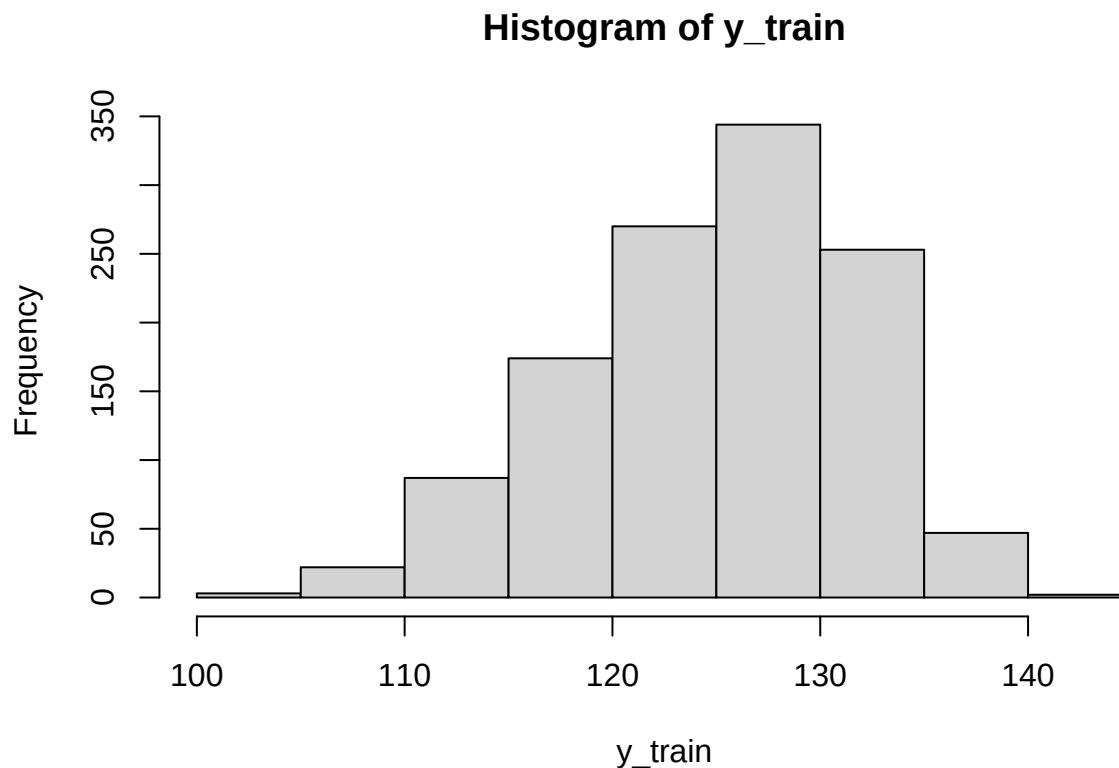
Table 7: Pairwise correlation matrix among predictors

	frequency	attack_angle	chord_length	free_stream_velocity	suction_side_displacement_thickness
frequency	1.00	-0.28	0.01	0.14	-0.24
attack_angle	-0.28	1.00	-0.50	0.07	0.76
chord_length	0.01	-0.50	1.00	0.01	-0.22
free_stream_velocity	0.14	0.07	0.01	1.00	0.00
suction_side_displacement_thickness	-0.24	0.76	-0.22	0.00	1.00



Predictor-predictor correlations are weak across most pairs. The one exception is suction_side_displacement_thickness and attack_angle, correlated at $r \approx 0.76$ - a moderate relationship, plausible given both describe the airfoil's boundary-layer geometry at the trailing edge. No other pair exceeds $|r| \approx 0.5$. This single moderate correlation is revisited with the VIF check below and is the main multicollinearity signal this dataset offers ridge and lasso a reason to address in Problem 2.

1.3.2 Response Distribution



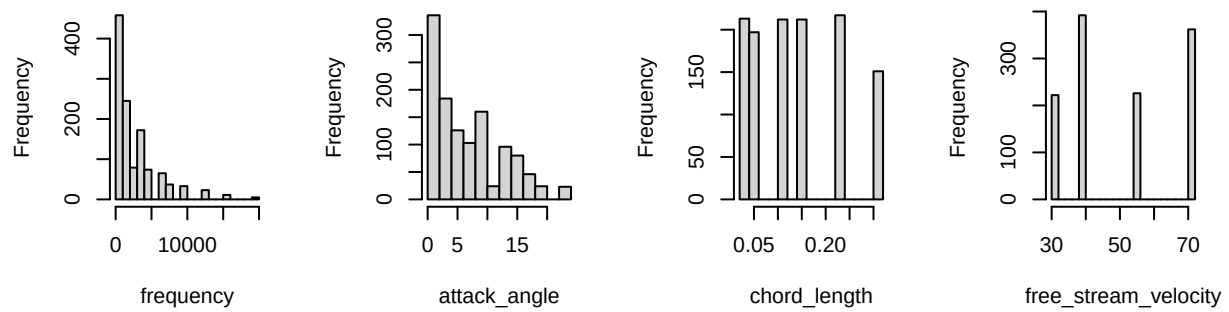
Row	Value
Min.	103.3800
1st Qu.	120.1900
Median	125.6385
Mean	124.8073
3rd Qu.	130.0312
Max.	140.9870

The response ranges from 103.4 to 141 dB with a median of 125.6 dB and mean 124.8 dB. The histogram and the small gap between mean and median show a mild left skew (skewness ≈ -0.42): most measurements cluster in the 120- 130 dB range, with a longer tail toward the quieter (lower-dB) end rather than the loud end. The skew is mild enough that no response transformation is applied - RMSE and MAE on the original dB scale remain directly interpretable as sound-pressure prediction errors, which matters for Problem 1’s evaluation step.

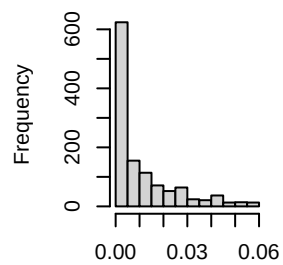
1.3.3 Predictor Histogram & Boxplot

1. Histogram

Distribution of frequency Distribution of attack_angle Distribution of chord_length Distribution of free_stream_velocity

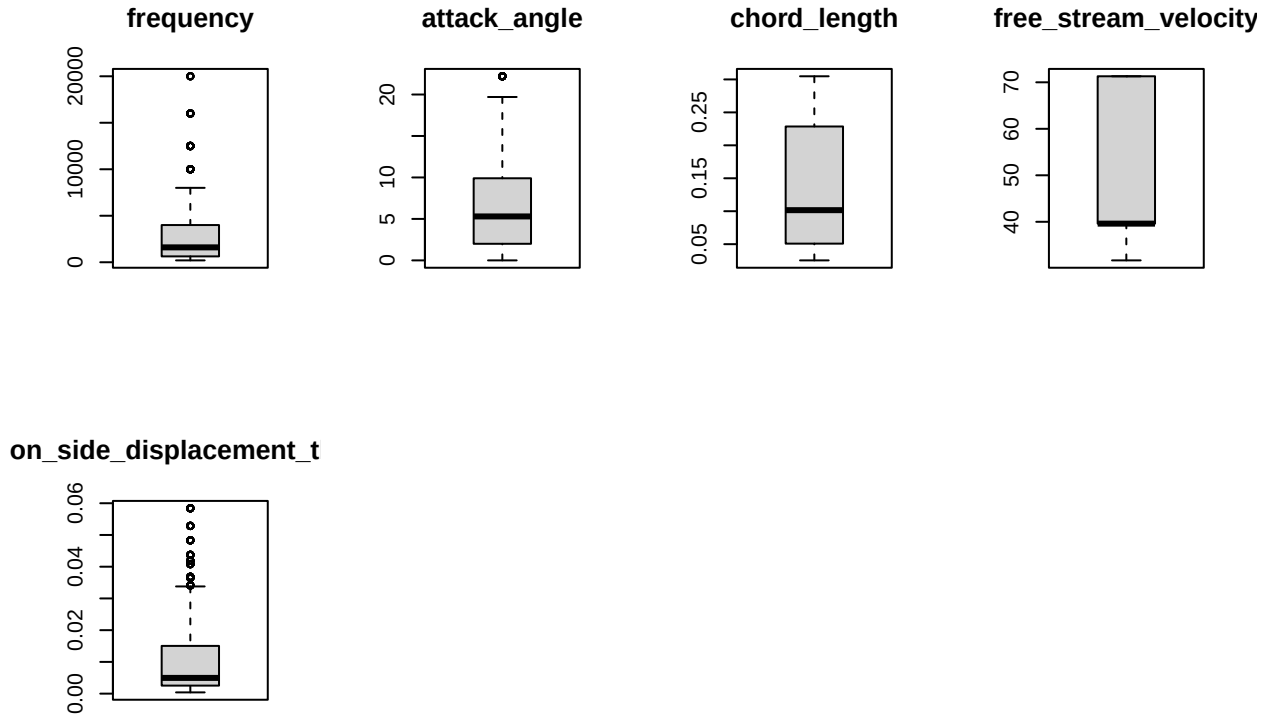


of suction_side_displacement_thick



suction_side_displacement_thick

2. Boxplot



The histograms and boxplots make the 1A distinct-value counts visible. `chord_length` and `free_stream_velocity` appear as 6 and 4 narrow, evenly spaced spikes rather than a smooth spread, and `frequency` shows 21 spikes concentrated at the low end with a long right tail - these are the fixed experimental levels described in 1A, not gaps in the data. `attack_angle` and `suction_side_displacement_thickness` are closer to continuous, right-skewed shapes, matching the skewness values quantified in 1B. The boxplots' upper whiskers and flagged points for these two variables are the same training-set values already counted in the 1B outlier check, retained for the same reason: they are valid high-frequency / high-angle test conditions, not data errors.

1.3.4 Correlation between predictors and response

Table 8: Correlation between predictors and scaled_sound_pressure

	scaled_sound_pressure
frequency	-0.40
attack_angle	-0.16
chord_length	-0.23
free_stream_velocity	0.11
suction_side_displacement_thickness	-0.30

Every predictor's marginal correlation with the response is weak in magnitude (all $|r| < 0.4$). fre-

quency is the strongest single linear predictor ($r \approx -0.4$), followed by `suction_side_displacement_thickness`; `free_stream_velocity` is weakest and the only predictor positively correlated with the response. No individual predictor comes close to explaining the response on its own, which is consistent with the assignment brief's description of this dataset as having “few features but nonlinear relationships” - a linear combination of all five predictors, rather than any single one, is needed, and Problem 2's regularized models are expected to combine these weak individual signals rather than select one dominant predictor and discard the rest.

1.3.5 Check multicollinearity using VIF

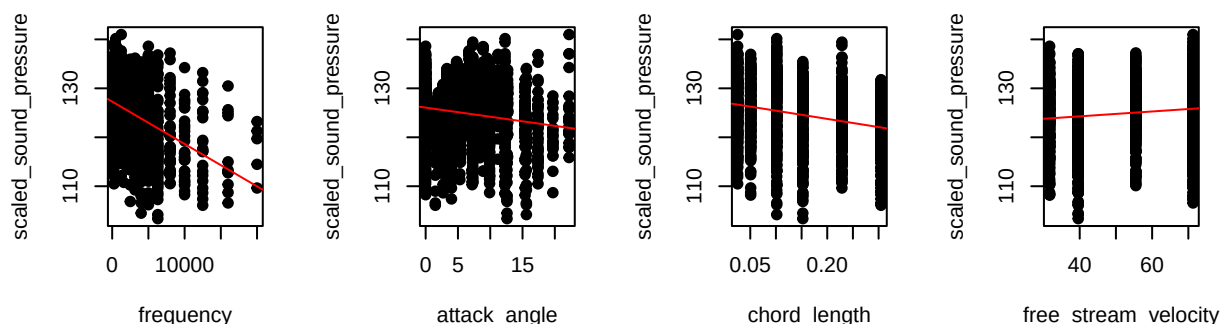
Table 9: Variance Inflation Factor (VIF)

	VIF
frequency	1.14
attack_angle	3.48
chord_length	1.50
free_stream_velocity	1.05
suction_side_displacement_thickness	2.58

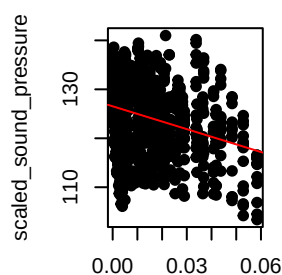
Every VIF is comfortably below the conventional threshold of 5, with `attack_angle` the highest at 3.48. This confirms the correlation-matrix finding above: the airfoil predictors share only mild, tolerable overlap, so no predictor needs to be dropped for redundancy, and OLS in Problem 2 is not expected to suffer severe coefficient instability from multicollinearity. The moderate `attack_angle` / `suction_side_displacement_thickness` correlation is still the largest source of shared variance in the data and remains worth tracking through ridge and lasso, but it falls well short of the kind of collinearity ($\text{VIF} \gg 5$) that would make variable selection a necessity rather than a convenience.

1.3.6 Predictor-response plots

frequency vs scaled_sound_pressure, attack_angle vs scaled_sound_pressure, chord_length vs scaled_sound_pressure, free_stream_velocity vs scaled_sound_pressure



suction_side_displacement_thickness vs scaled_sound_pressure



The fitted red lines are visibly weak for every panel - the same $|r| < 0.4$ ceiling seen in the correlation table above - but the point clouds are not simply noise around those lines. The **frequency** panel in particular shows the response bending down over its range rather than following the straight line, echoing the linear-vs-quadratic R^2 comparison from 1B: the straight-line fit misses curvature that a richer model could capture. The **chord_length** and **free_stream_velocity** panels show their points arranged in 6 and 4 vertical strips (their fixed experimental levels), each strip itself spanning a wide range of the response - visual confirmation that these two variables alone say little about the outcome, and any predictive power must come from combining all five predictors together, which is exactly what Problem 2's models are built to do.

1.3.7 Question

Taken together, the checks in 1A-1C describe a clean but weak-signal regression problem, and that combination shapes what the rest of Problem 1 needs to prioritise.

Data quality is not the challenge here. There are no missing values, no duplicated rows, and the “outliers” flagged by the $1.5 \times \text{IQR}$ rule are legitimate wind-tunnel test conditions from a controlled experiment, not data-entry errors - so cleaning required no row deletions or imputation, only the decision (1B) to retain every training row and to skip a log transform after checking that it does not help the linear fit.

The predictors are almost uncorrelated with each other. All VIFs sit well under 5, and

the one moderately correlated pair (`attack_angle` / `suction_side_displacement_thickness`, $r \approx 0.76$) falls short of severe multicollinearity. This means ridge and lasso in Problem 2 are not being asked to rescue an ill-conditioned design matrix the way they would with, say, several near-duplicate predictors; their main job here is tuning the bias-variance trade-off and (for lasso) testing whether any predictor can be dropped outright, rather than resolving instability.

The predictors are individually weak but the relationships are nonlinear. Every predictor-response correlation is under 0.4 in magnitude, and no single predictor separates high- from low-noise conditions on its own (the `chord_length` and `free_stream_velocity` panels above make this explicit, with each fixed level covering a wide spread of the response). At the same time, `frequency` fits a quadratic term noticeably better than a straight line (1B), and three of the five predictors are effectively small sets of experimental design levels (1A) rather than smooth continuous measurements. Both point to the same conclusion the assignment brief anticipates for this dataset: the frequency-response relationship, and likely others, are genuinely nonlinear, so a **linear baseline (OLS) is expected to leave a meaningful amount of predictable structure on the table**, and its residuals in Problem 2 should be inspected for exactly this kind of unmodelled curvature rather than assumed to be pure noise.